

Quantum Structures as Generative Scores: Partition Logic, Generative Logic, and Aesthetic Form

Christian Jendreiko 

HSD University of Applied Sciences, Münsterstraße 156, D-40476 Düsseldorf, Germany*

Karl Svozil 

Institute for Theoretical Physics, TU Wien, Wiedner Hauptstrasse 8-10/136, A-1040 Vienna, Austria†

(Dated: September 25, 2026)

We connect partition logic with Generative Logic by translating finite partition logics into Prolog-based Simple Generative Logic Grammars. As a proof of concept, we use the five-atom V-logic L_{12} to generate a modular visual artifact, the *Quantum Square*. The approach separates logical structure from its visual, textual, or sonic realization. The examples establish a generative design method and suggest a way to introduce contexts and complementarity through perceptible form.

Keywords: partition logics, logic programming, quantum structures, quantum music, generative logic, complementarity, generative art, generative design, science communication, Prolog, education

I. INTRODUCTION

This paper connects two lines of work that are rarely discussed together. One originates in the logical and combinatorial study of finite empirical structures in quantum foundations, especially partition logics and related event structures [1–5]. The other originates in artistic research and design education, namely Christian Jendreiko’s concept of *Generative Logic* and the associated *Simple Generative Logic Grammar* (SGLG), which treats Prolog as a generative engine for aesthetic production [6, 7].

The shared premise is that formal structures are not only objects of abstract analysis; they can also act as structuring agents. In mathematics, this idea is familiar from fractals, cellular automata, L-systems, symmetry groups, and stochastic processes. In design, one encounters related practices whenever a rule system is used to generate, constrain, or organize visual, textual, or sonic material. The present paper asks whether partition logics can be used in this way: not merely as representations of finite event structures, but as engines for the generation of perceptible artifacts.

This question has both an aesthetic and a communicative side. Finite logical structures exhibit repetition, asymmetry, recurrence, and nontrivial forms of overlap. These can become aesthetically productive once mapped into color, shape, spacing, rhythm, or timbre. At the same time, concepts such as context, complementarity, and non-Boolean composition are often difficult to grasp when they remain purely symbolic. If they can be externalized into visual or sonic form without losing their structural discipline, they may become more accessible.

The key methodological move of the paper is the separation of three layers:

1. a *formal skeleton*, given by a partition logic or related finite event structure;

2. a *generative grammar*, which translates that skeleton into a structured template of positions;
3. a *sign repertoire and rendering rule*, which realizes that structure in a concrete medium.

This separation is important both artistically and scientifically. Artistically, it allows the same structure to migrate across media. Scientifically, it prevents us from conflating a combinatorial structure with a particular physical realization or probability theory.

The paper does four things. First, it gives a compact account of the partition-logical source structures used here. Second, it states a general translation from finite partition logics with separating sets of two-valued states into SGLGs. Third, it develops a proof of concept: the V-logic L_{12} is translated into a visual artifact, the *Quantum Square*. Fourth, it extends the method to a triangle logic and discusses the resulting structures from the perspectives of design and science communication.

II. PARTITION LOGIC BETWEEN FORMAL STRUCTURE AND DESIGN MATERIAL

A. Partitions and partition logics

Let $\Omega_n = \{1, 2, \dots, n\}$ be a finite set. A *partition* π of Ω_n is a family of nonempty, pairwise disjoint subsets whose union is Ω_n . The elements of a partition are its *blocks*. Each partition can be identified with the atoms of a finite Boolean algebra. A *partition logic* is obtained by selecting several such Boolean algebras and pasting them together through identified common elements [2, 3, 5].

In the language of quantum logic, the pasted Boolean algebras are often called *contexts*. They provide local classical views within a structure that may be non-Boolean. Here, complementarity means that no single available context jointly resolves the distinctions offered by the different contexts. Although partitions of the same set always have a common set-theoretic refinement, that refinement need not be an available

* christian.jendreiko@hs-duesseldorf.de

† karl.svozil@tuwien.ac.at

measurement. The distinction is therefore between what exists in the underlying state space and what can be resolved within the chosen empirical structure.

Partition logics are therefore well suited to interdisciplinary transfer. They are finite, explicit, and combinatorial. At the same time, they already carry key motifs associated with quantum theory: contextual decomposition, overlap of contexts, and non-Boolean composition.

B. Formal Example A: a horizontal sum of binary partitions

A minimal example uses the set

$$\Omega_3 = \{1, 2, 3\}. \quad (1)$$

Consider the three binary partitions

$$\begin{aligned} \pi_1 &= \{\{1\}, \{2, 3\}\}, \\ \pi_2 &= \{\{2\}, \{1, 3\}\}, \\ \pi_3 &= \{\{3\}, \{1, 2\}\}. \end{aligned} \quad (2)$$

If these three Boolean algebras are pasted only in their bottom and top elements, one obtains a horizontal sum of three four-element Boolean algebras. This is a simple non-Boolean event structure.

To make the example more readable, let us name the six atoms

$$\begin{aligned} p &\leftrightarrow \{1\}, & \neg p &\leftrightarrow \{2, 3\}, \\ q &\leftrightarrow \{2\}, & \neg q &\leftrightarrow \{1, 3\}, \\ r &\leftrightarrow \{3\}, & \neg r &\leftrightarrow \{1, 2\}. \end{aligned} \quad (3)$$

A *two-valued state* assigns each atom either 0 or 1, with exactly one atom assigned 1 in each context. A family of states is *separating* if any two distinct propositions receive different values in at least one state. The underlying points 1, 2, 3 induce three such states s_1, s_2, s_3 . They form a separating selection from the eight two-valued states of the abstract horizontal sum; we use this selection for the example. The supports are then

$$\begin{aligned} T(p) &= \{s_1\}, & T(\neg p) &= \{s_2, s_3\}, \\ T(q) &= \{s_2\}, & T(\neg q) &= \{s_1, s_3\}, \\ T(r) &= \{s_3\}, & T(\neg r) &= \{s_1, s_2\}. \end{aligned} \quad (4)$$

Even this small logic shows a pattern that later becomes generative: the same finite set of state symbols is redistributed from context to context.

C. Formal Example B: the V-logic L_{12}

Our main proof of concept uses the five-atom V-logic L_{12} , consisting of two three-atomic contexts

$$C_1 = \{a, b, c\}, \quad C_2 = \{c, d, e\}, \quad (5)$$

with intertwining atom c . Figure 1(a) shows the corresponding 3-uniform hypergraph in Greechie-style notation [8], in which contexts are represented schematically by unbroken lines that intersect at shared atoms.

This logic has exactly five two-valued states, shown in Table I. They separate the atoms and therefore provide a classical state space for a partition-logical representation [9, Theorem 0].

The supports of the atoms are

$$\begin{aligned} T(a) &= \{s_1, s_2\}, & T(b) &= \{s_3, s_4\}, & T(c) &= \{s_5\}, \\ T(d) &= \{s_2, s_4\}, & T(e) &= \{s_1, s_3\}. \end{aligned} \quad (6)$$

Equivalently, by identifying each atom with the set of two-valued states in which it is assigned the value 1 [5], the two contexts are represented by the partitions

$$\begin{aligned} &\{\{s_1, s_2\}, \{s_3, s_4\}, \{s_5\}\}, \{\{s_5\}, \{s_2, s_4\}, \{s_1, s_3\}\}; \\ &\text{or, equivalently,} \\ &\{\{1, 2\}, \{3, 4\}, \{5\}\}, \{\{5\}, \{2, 4\}, \{1, 3\}\}. \end{aligned} \quad (7)$$

The conceptual importance of this example, as well as the previous example involving horizontal sums, is that it exhibits complementarity while still possessing a separating set of two-valued states. Thus it does *not* exhibit Kochen-Specker-type value indefiniteness [9]. For science communication this is advantageous: it allows one to explain context dependence without prematurely introducing stronger no-go results.

D. A quantum realization of the same logical form

The same V-logic also admits a faithful orthogonal representation [10, 11] in three-dimensional Hilbert space. A simple choice is

$$\begin{aligned} |a\rangle &= (1, 0, 0)^T, & |b\rangle &= (0, 1, 0)^T, \\ |c\rangle &= (0, 0, 1)^T, \\ |d\rangle &= (\cos \theta, \sin \theta, 0)^T, & |e\rangle &= (-\sin \theta, \cos \theta, 0)^T, \end{aligned} \quad (8)$$

with $0 < \theta < \pi/2$. Then $\{|a\rangle, |b\rangle, |c\rangle\}$ and $\{|c\rangle, |d\rangle, |e\rangle\}$ form two orthonormal bases sharing the ray c . Neither $|a\rangle$ nor $|b\rangle$ is orthogonal to $|d\rangle$ or $|e\rangle$.

The V-logic thus provides both a partition representation and a quantum realization. The latter also requires geometric information, here the angle θ , that the logical skeleton alone does not specify. The grammars below use the two-valued-state supports and introduce no probabilistic weighting.

TABLE I. The five two-valued states of the V-logic L_{12} .

state	a	b	c	d	e
s_1	1	0	0	0	1
s_2	1	0	0	1	0
s_3	0	1	0	0	1
s_4	0	1	0	1	0
s_5	0	0	1	0	0

III. GENERATIVE LOGIC AND SGLGS

A. Generative Logic

Generative Logic, introduced in Refs. [6, 7], treats Prolog not only as a language for symbolic computation but as a generative environment in which rules, substitutions, and derivations become design operations. The Prolog inference engine functions as a disciplined producer of variants. In this perspective, the logical formula is not only a specification of truth conditions; it is also a compositional device.

This is particularly relevant in artistic research and education. It allows students and researchers to work with rule systems that remain both transparent and executable. The output need not remain verbal or symbolic. Once a derivation is mapped into color, sound, or spatial arrangement, a logical process becomes a perceptible artifact.

B. A Simple Generative Logic Grammar

To turn a partition logic into an executable generative scheme, we use a Simple Generative Logic Grammar (SGLG) [7]. In the present paper, this grammar is understood in a practical way: it specifies how a finite artifact is assembled from rows and symbols, while a separate rendering map determines how those symbols appear in a given medium.

We write the grammar as

$$G = (\mathcal{N}, \Sigma, P, S, M, \Lambda), \quad (9)$$

where $\mathcal{N}$ is a set of nonterminal symbols, Σ is a set of state symbols, P is a set of production rules, S is the start symbol, and Λ contains layout symbols such as separators and line breaks. Productions expand nonterminals into sequences over $\mathcal{N} \cup \Sigma \cup \Lambda$, while M assigns concrete realizations to the terminal and layout symbols.

For the present purposes, a production rule of the form

$$\text{Head} \rightarrow \text{Body} \quad (10)$$

may be read simply as an assembly rule: the head is expanded into the ordered sequence given by the body. For example, if the complete artifact consists of five rows, one may write

$$q_L \rightarrow abcde \quad (11)$$

meaning that the whole object is built from the rows a, b, c, d, e . A row may then be specified by a rule such as

$$a \rightarrow s_1 s_2 br s_3 s_4 s_5 n \quad (12)$$

where $s_1, \dots, s_5$ are state symbols, br is a separator, and n denotes a line break.

The grammar therefore determines the structure of the output, but not yet its concrete appearance. The rendering map M assigns a medium-specific realization to each symbol. For instance, one may choose

$$s_1 \mapsto \text{green square}, \quad s_2 \mapsto \text{blue square} \quad (13)$$

and similarly for the other symbols. By changing M , the same grammar can be realized visually, textually, or sonically.

In Jendreiko's broader framework, such rules may also be interpreted as containment relations, in that larger forms contain the smaller ones they are composed of.

The non-recursive character of the grammar is important here. It guarantees that every derivation terminates after finitely many steps and yields a finite, inspectable artifact.

IV. FROM PARTITION LOGIC TO GRAMMAR

The preceding grammar scheme becomes concrete once its symbols are taken from a partition logic. The basic idea is simple: atoms of the logic become row labels, two-valued states become recurring symbols, and a separator divides the states that make a given atom true from those that make it false.

Let L be a finite partition logic with atoms

$$A = \{x_1, \dots, x_m\} \quad (14)$$

and a separating set of two-valued states

$$V_L = \{v_1, \dots, v_N\}. \quad (15)$$

Associate a symbol s_i with each two-valued state v_i , and write

$$\mathcal{S}_N = \{s_1, \dots, s_N\}. \quad (16)$$

For each atom x_j , define

$$T(x_j) = \{s_i \in \mathcal{S}_N \mid v_i(x_j) = 1\}, \quad F(x_j) = \mathcal{S}_N \setminus T(x_j). \quad (17)$$

Thus $T(x_j)$ is the set of state symbols for which x_j is true, and $F(x_j)$ is the complementary set for which x_j is false. Fix an order of the atoms and of the states. In a production rule, $T(x_j)$ and $F(x_j)$ denote lists in the induced state order; the examples use increasing indices.

We then construct a grammar G_L with start symbol q_L by setting

$$q_L \rightarrow x_1 x_2 \cdots x_m, \quad (18)$$

$$x_j \rightarrow T(x_j) br F(x_j) n. \quad (19)$$

Here br is a separator symbol and n is a layout symbol such as a line break. In words: the whole artifact consists of one row for each atom, and the row of x_j lists first those states in which x_j is true and then those in which it is false.

Proposition 1. *Let L be a finite partition logic with separating two-valued states. The grammar G_L constructed from Eqs. (17)–(19) preserves the incidence relation between atoms and two-valued states. In the generated row for x_j , the symbol s_i occurs to the left of the separator iff $v_i(x_j) = 1$, and to the right iff $v_i(x_j) = 0$.*

Proof. The two lists partition the state symbols according to the value of $v_i(x_j)$, and the row rule places them on opposite sides of the separator. Concatenating all rows retains this information for every atom. $\square$

The translation preserves atom–state incidence for the selected state family; the designated contexts remain part of the source specification and are not separately encoded in the rows. To recover the incidence from a rendering, the state symbols, separator, and row boundaries must remain distinguishable. Choosing a different separating family or ordering can change the artifact, so these choices belong to the score.

The construction is easiest to understand through examples.

A. Formal Example C: the horizontal sum as a grammar

Applying the translation to the horizontal-sum example of Eq. (2) yields

$$\begin{aligned} p &\rightarrow s_1 br s_2 s_3 n, & \neg p &\rightarrow s_2 s_3 br s_1 n, \\ q &\rightarrow s_2 br s_1 s_3 n, & \neg q &\rightarrow s_1 s_3 br s_2 n, \\ r &\rightarrow s_3 br s_1 s_2 n, & \neg r &\rightarrow s_1 s_2 br s_3 n. \end{aligned} \quad (20)$$

Each row corresponds to one atom, and the recurring symbols s_1, s_2, s_3 record which of the three selected two-valued states support that atom. Even in this small example, the grammar already produces a visible pattern of repetition and redistribution.

B. Formal Example D: the V-logic as a grammar

The V-logic L_{12} is handled in exactly the same way. Since it has five atoms and five two-valued states, the resulting artifact has five rows built from the recurring symbols $s_1, \dots, s_5$. Using the supports listed in Eq. (6), we obtain

$$\begin{aligned} a &\rightarrow s_1 s_2 br s_3 s_4 s_5 n, \\ b &\rightarrow s_3 s_4 br s_1 s_2 s_5 n, \\ c &\rightarrow s_5 br s_1 s_2 s_3 s_4 n, \\ d &\rightarrow s_2 s_4 br s_1 s_3 s_5 n, \\ e &\rightarrow s_1 s_3 br s_2 s_4 s_5 n. \end{aligned} \quad (21)$$

This is the structural core of the *Quantum Square*. The grammar fixes the arrangement of symbols row by row, beginning with the start rule as the point of entry for the generation:

$$q_L \rightarrow abcde. \quad (22)$$

To obtain a visual realization, we map the five state symbols to five squares of identical size with five colors:

$$\begin{aligned} s_1 &\mapsto \text{green}, & s_2 &\mapsto \text{blue}, & s_3 &\mapsto \text{red}, \\ s_4 &\mapsto \text{orange}, & s_5 &\mapsto \text{violet}. \end{aligned} \quad (23)$$

The separator br is rendered as a black square. The name *Quantum Square* refers to the artwork; with separators included, the displayed arrangement has five rows and six columns.

The supports determine which symbols lie on each side of the separator; the chosen ordering determines their sequence. The palette and tile geometry are rendering choices.

V. THE QUANTUM SQUARE: STRUCTURE AND RENDERINGS

Figure 1 connects the V-logic and its incidence schema with two renderings of the same grammar: colored tiles and a compact typographic arrangement.

Panels (c) and (d) preserve the same incidence information while offering different perceptual cues: color emphasizes recurrence, whereas digits make symbol identity explicit. They also show the scope of the method: a fixed grammar produces one symbolic arrangement, while changes to its rendering yield different realizations.

A. Why this object is aesthetically interesting

The *Quantum Square* is not a picture of a quantum system in any naive representational sense. It does not depict particles, trajectories, or apparatuses. Rather, it externalizes a formal relation as a visible organization of repetition and difference.

What becomes aesthetically salient is the disciplined reappearance of the same five colors across distinct rows, each row embodying a different logical perspective on the same finite state space. In design terms, this produces a tension between consistency and redistribution. In humanities terms, one may say that the logic functions as a score: it does not determine one unique expressive surface, but it constrains a field of possible realizations. In logical terms, it preserves the incidence of atoms and the selected states.

VI. A TRIANGLE-LOGIC GENERALIZATION OF THE QUANTUM SQUARE

To show that the method is not specific to the V-logic, we now apply it to the *Wright triangle* [3, 12], represented by three partitions:

$$\begin{aligned} \pi_1 &= \{\{1\}, \{2, 3\}, \{4\}\}, \\ \pi_2 &= \{\{4\}, \{1, 2\}, \{3\}\}, \\ \pi_3 &= \{\{3\}, \{2, 4\}, \{1\}\}, \\ \Pi_\Delta &= \{\pi_1, \pi_2, \pi_3\}. \end{aligned} \quad (24)$$

This structure is again a partition logic, but now with three contexts arranged cyclically rather than two contexts joined in a V-shape.

To make the logic easier to read, let us name its six atoms by

$$\begin{aligned} a &\leftrightarrow \{1\}, & b &\leftrightarrow \{2, 3\}, & c &\leftrightarrow \{4\}, \\ d &\leftrightarrow \{1, 2\}, & e &\leftrightarrow \{3\}, & f &\leftrightarrow \{2, 4\}. \end{aligned} \quad (25)$$

The three contexts are then

$$C_1 = \{a, b, c\}, \quad C_2 = \{c, d, e\}, \quad C_3 = \{e, f, a\}. \quad (26)$$

Thus a , c , and e are the intertwining atoms: each belongs to two contexts, while b , d , and f belong to only one.

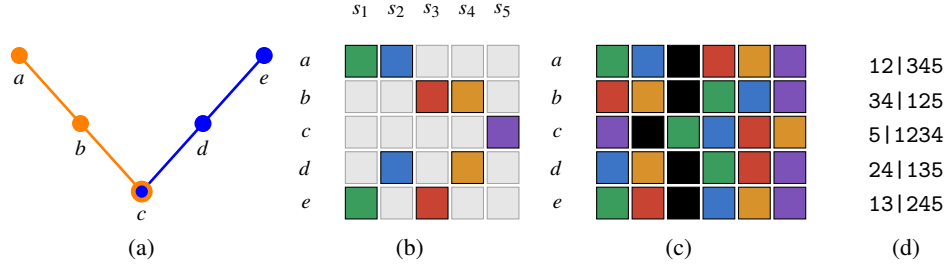


FIG. 1. The V-logic L_{12} . (a) Greechie-style hypergraph. (b) Atom-state incidence schema, with colored cells indicating $v_i(x) = 1$ and gray cells indicating $v_i(x) = 0$. (c) The grammar-generated output, the *Quantum Square*. In each row, the states assigning value 1 to the atom are listed first, then a black separator, and then the states assigning value 0. (d) The same rows rendered as digits, with | as separator.

TABLE II. The four two-valued states of the triangle logic.

state	a	b	c	d	e	f
s_1	1	0	0	1	0	0
s_2	0	1	0	1	0	1
s_3	0	1	0	0	1	0
s_4	0	0	1	0	0	1

Unlike the V-logic, this structure cannot be realized by six distinct rays forming these three orthonormal bases in three dimensions. The corner rays a, c, e would already form a basis, forcing $b = e, d = a$, and $f = c$ as rays. The example therefore extends the partition-logical construction, not the preceding quantum realization.

This logic has exactly four two-valued states, shown in Table II. As in the V-logic case, these states separate the atoms and therefore provide a classical state space for a partition-logical representation. Writing these states as s_1, s_2, s_3, s_4 , the supports of the atoms are read off directly from Eq. (25):

$$\begin{aligned} T(a) &= \{s_1\}, & T(b) &= \{s_2, s_3\}, & T(c) &= \{s_4\}, \\ T(d) &= \{s_1, s_2\}, & T(e) &= \{s_3\}, & T(f) &= \{s_2, s_4\}. \end{aligned} \quad (27)$$

Equivalently, by identifying each atom with the set of two-valued states in which it is assigned the value 1, the three contexts are represented by the partitions

$$\begin{aligned} &\{\{s_1\}, \{s_2, s_3\}, \{s_4\}\}, \\ &\{\{s_4\}, \{s_1, s_2\}, \{s_3\}\}, \\ &\{\{s_3\}, \{s_2, s_4\}, \{s_1\}\}. \end{aligned} \quad (28)$$

Applying the translation scheme of the previous section yields the start rule

$$q_{\Delta} \rightarrow abcdef \quad (29)$$

and the row rules

$$\begin{aligned} a &\rightarrow s_1 br s_2 s_3 s_4 n, \\ b &\rightarrow s_2 s_3 br s_1 s_4 n, \\ c &\rightarrow s_4 br s_1 s_2 s_3 n, \\ d &\rightarrow s_1 s_2 br s_3 s_4 n, \\ e &\rightarrow s_3 br s_1 s_2 s_4 n, \\ f &\rightarrow s_2 s_4 br s_1 s_3 n. \end{aligned} \quad (30)$$

Figure 2 shows the triangle hypergraph, its incidence schema, and its tile rendering. The same row construction now uses six atoms and four state symbols, extending the design from two contexts to a cyclic arrangement of three.

Compared to the V-logic, the triangle uses fewer colors across more rows. Its different support pattern changes both the recurrence of colors and the positions of the separators.

VII. PROLOG REALIZATIONS

The following complete SWI-Prolog programs render state symbols as digits and the separator as |. Double-quoted terminals in a DCG produce character codes; [10] supplies a newline. The companion files contain both programs and an SVG renderer that maps their output to the palettes and tile arrangements in Figs. 1(c) and 2(c).

A. V-logic code

Save Listing 1 as `v-logic.pl`. Load it with `use_module('v-logic.pl', []).` and run `v_logic:output`. The result is the typographic rendering in Fig. 1(d).

B. Triangle-logic code

Save Listing 2 as `triangle-logic.pl`. Load it with `use_module('triangle-logic.pl', []).` and run `triangle_logic:output`. The separate modules allow both examples to be used without mixing their row rules.

For colored output, load `render-tableaux.pl` and run `render_tableaux:render_all`. This writes one SVG file

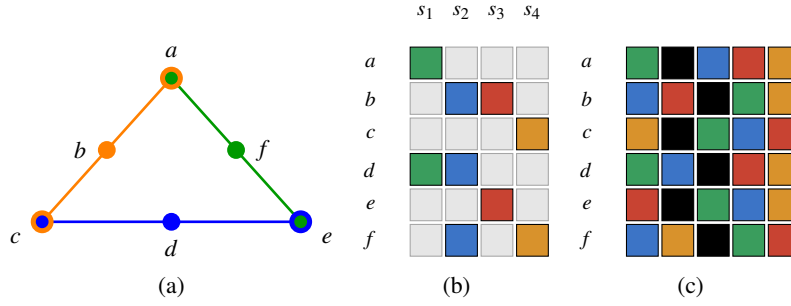


FIG. 2. Triangle-logic generalization of the *Quantum Square*. (a) Greechie-style depiction of the triangle logic associated with Eq. (24). The intertwining atoms a , c , and e are shown by concentric circles because each belongs to two contexts. (b) Atom-state incidence schema. (c) Tile rendering obtained from the grammar in Eq. (30), using four recurring state colors and a black separator between support and complement. Each row lists the states with value 1, then the separator, and then the states with value 0.

Listing 1. Complete V-logic program with a typographic repertoire.

```
% SWI-Prolog: typographic V-logic tableau.
:- module(v_logic, [v_logic//0, output/0]).

% Structural layer
v_logic --> a,b,c,d,e.
a --> s1,s2,br,s3,s4,s5,n.
b --> s3,s4,br,s1,s2,s5,n.
c --> s5,br,s1,s2,s3,s4,n.
d --> s2,s4,br,s1,s3,s5,n.
e --> s1,s3,br,s2,s4,s5,n.

% Typographic repertoire and layout
s1 --> "1".
s2 --> "2".
s3 --> "3".
s4 --> "4".
s5 --> "5".
br --> "|".
n --> [10].

% Rendering layer
output :-
    phrase(v_logic, Codes),
    format("~s", [Codes]).
```

Listing 2. Complete triangle-logic program.

```
% SWI-Prolog: typographic Wright triangle.
:- module(triangle_logic,
    [triangle_logic//0, output/0]).

% Structural layer
triangle_logic --> a,b,c,d,e,f.
a --> s1,br,s2,s3,s4,n.
b --> s2,s3,br,s1,s4,n.
c --> s4,br,s1,s2,s3,n.
d --> s1,s2,br,s3,s4,n.
e --> s3,br,s1,s2,s4,n.
f --> s2,s4,br,s1,s3,n.

% Typographic repertoire and layout
s1 --> "1".
s2 --> "2".
s3 --> "3".
s4 --> "4".
br --> "|".
n --> [10].

% Rendering layer
output :-
    phrase(triangle_logic, Codes),
    format("~s", [Codes]).
```

for each example. The rendering instructions are kept in the companion file so that the structural grammars remain easy to inspect.

VIII. HUMANITIES- AND DESIGN-ORIENTED INTERPRETATION

A. Structure as score

From a design perspective, the partition logic can be interpreted as a score. It is not yet the finished artifact, just as a musical score is not yet a performance. But it prescribes a disciplined arrangement of possibilities. The grammar derived from the logic functions as a notation of relations, recursences, and separations.

This score-like character is important because it avoids two

extremes. On the one hand, it is more rigid than unconstrained improvisation: the formal structure cannot simply be ignored. On the other hand, it is less rigid than a single finished image: many realizations remain possible. The logic therefore occupies a productive middle ground between rule and freedom.

B. Structural template and sign repertoire

A simple teaching exercise is to compare the two renderings in Fig. 1, then change the palette while keeping the rows fixed. A second exercise keeps the palette fixed and changes the logical source. Such comparisons may help students distinguish structural constraints from rendering choices; their educational effectiveness remains to be evaluated.

The same V-logic may therefore become:

- a colored tile image,
- a typographic grid,
- a sequence of syllables,
- a rhythmic or harmonic pattern,
- or a data-driven animation.

Beyond the visual examples presented here, a sonic prototype has been developed by Christian Jendreiko in collaboration with the composer Klaus Roeder. Further work in this direction is being pursued by a research group led by Christian Jendreiko at HSD University of Applied Sciences in Düsseldorf. The group includes Klaus Roeder, Björn Lellmann, and Robert Eisinger.

C. From diagram to perceptual offering

In science communication, it is often tempting to regard visualization as a transparent window onto a concept. Our approach suggests a more careful view. The generated object is not a transparent copy of the logic. It is a *perceptual offering*: a structured artifact that affords certain acts of recognition, comparison, and reflection.

The value of such artifacts lies partly in their legibility and partly in their sensuous force. A purely formal matrix may be clearer analytically; a more aesthetic tile arrangement may be more memorable or engaging. These are not mutually exclusive aims, but they are distinct. The tension between them should be treated as a design problem rather than suppressed.

D. Complementarity without overstatement

The V-logic permits a staged introduction to complementarity: the hypergraph shows distinct contexts and their overlap, while the state table makes the existence of global truth assignments explicit. Kochen-Specker configurations without such assignments require a further step.

The tile image can support this comparison, provided that the source contexts are shown alongside it. Whether this improves understanding is an empirical question.

IX. DISCUSSION

Several points deserve emphasis.

First, the construction is not limited to the V-logic. Any finite partition logic with separating two-valued states can be translated into an SGLG in the manner described above. Larger examples, including pentagon logics and more intricate pasted structures, should yield richer aesthetic and pedagogical possibilities [5, 12].

Second, not every finite quantum structure admits a separating set of two-valued states, and not every partition logic admits the kind of quantum realization given for the V-logic. The Wright triangle illustrates the latter distinction. Structures without suitable state supports would need a different translation, perhaps based on their context incidence or orthogonal representations.

Third, the distinction between logical skeleton and repertoire is not merely a convenience of notation. It is what allows one and the same source structure to appear across media and to function both as a formal object and as a generative score.

Fourth, the method invites empirical investigation. Which mappings are most understandable to non-specialist audiences? Which visual or sonic forms best communicate overlap of contexts? When does aesthetic richness enhance understanding, and when does it obscure it? These are open interdisciplinary research questions.

X. CONCLUSION

We have proposed and exemplified a concrete bridge between partition logic and Generative Logic. The bridge consists of a translation from finite partition logics with separating two-valued states into simple non-recursive grammars, together with a principled separation between formal skeleton, sign repertoire, and rendering layer.

Using the V-logic L_{12} , we derived a modular visual artifact, the *Quantum Square*, and a typographic rendering of the same grammar. We also gave additional formal examples, including a horizontal sum of binary partitions and a triangle-logic generalization.

The broader significance of the method is double. For design and artistic research, it opens a disciplined way of using logical structures as generative scores. For science communication, the combination of source diagrams and generated artifacts offers a way to introduce contexts and their overlap. The demonstrated result is precise: atom–state incidence can be translated into perceptible form, while the chosen states, ordering, and rendering remain explicit design decisions.

ACKNOWLEDGMENTS

Christian Jendreiko would like to thank François Fages, Thierry Marianne, and Guy Narboni for the invaluable insights gained during a collaborative working session in Paris. This research was funded in whole or in part by the *Austrian Science Fund (FWF)* [Grant DOI:10.55776/PIN5424624]. The authors acknowledge TU Wien Bibliothek for financial support through its Open Access Funding Programme.

-
- [1] K. Svozil, *Randomness & Undecidability in Physics* (World Scientific, Singapore, 1993).
 - [2] M. Schaller and K. Svozil, Partition logics of automata, *II Nuovo Cimento B* **109**, 167 (1994).
 - [3] A. Dvurečenskij, S. Pulmannová, and K. Svozil, Partition logics, orthoalgebras and automata, *Helvetica Physica Acta* **68**, 407 (1995), arXiv:1806.04271.
 - [4] K. Svozil, *Quantum Logic*, Discrete Mathematics and Theoretical Computer Science (Springer, Singapore, 1998).
 - [5] K. Svozil, Logical equivalence between generalized urn models and finite automata, *International Journal of Theoretical Physics* **44**, 745 (2005), arXiv:quant-ph/0209136.
 - [6] C. Jendreiko, Generative logic: Teaching Prolog as generative AI in art and design, in *Workshop Proceedings of the 40th International Conference on Logic Programming (ICLP-WS 2024)*, CEUR Workshop Proceedings, Vol. 3799, edited by J. Arias *et al.* (CEUR-WS.org, Dallas, TX, 2024).
 - [7] C. Jendreiko, The simple generative logic grammar: A tool for teaching logical thinking through visual research in art and design, in *Joint Proceedings of the Workshops and Doctoral Consortium of the 41st International Conference on Logic Programming (ICLP-WS-DC 2025)*, CEUR Workshop Proceedings, Vol. 4117, edited by D. Azzolini *et al.* (CEUR-WS.org, Rende, 2025).
 - [8] R. J. Greechie, Orthomodular lattices admitting no states, *Journal of Combinatorial Theory. Series A* **10**, 119 (1971).
 - [9] S. Kochen and E. P. Specker, The problem of hidden variables in quantum mechanics, *Journal of Mathematics and Mechanics* **17**, 59 (1967).
 - [10] L. Lovász, On the Shannon capacity of a graph, *IEEE Transactions on Information Theory* **25**, 1 (1979).
 - [11] M. Grötschel, L. Lovász, and A. Schrijver, Relaxations of vertex packing, *Journal of Combinatorial Theory, Series B* **40**, 330 (1986).
 - [12] R. Wright, Generalized urn models, *Foundations of Physics* **20**, 881 (1990).